Q1) This program simulates a snakes and ladders game using dictionaries and turn-based logic until a player reaches position 30.

```
@Farhan-Prog46 ➜ /workspaces/Data-Structures-and-Algorithms (main) $ /home/codespace/.python/current/bin/pytho
n /workspaces/Data-Structures-and-Algorithms/BTP500Group5-Assignment2/Question1_Muhammad_Umer.py
Welcome to Snakes and Ladders (First to reach 30 wins!)

Player 1's turn (Press Enter to roll dice).
Player 1 rolled a 6
Player 1 is now at position 6

Player 2's turn (Press Enter to roll dice).
Player 2 rolled a 5
Player 2 climbed a ladder from 5 to 8!
Player 2 is now at position 8

Player 1's turn (Press Enter to roll dice).
Player 1 rolled a 2
Player 1 is now at position 8

Player 2's turn (Press Enter to roll dice).
Player 2 rolled a 4
Player 2 is now at position 12

Player 1's turn (Press Enter to roll dice).
Player 1 rolled a 3
Player 1 climbed a ladder from 11 to 26!
Player 1 is now at position 26

Player 2's turn (Press Enter to roll dice).
Player 2 rolled a 4
Player 2 is now at position 16

Player 1's turn (Press Enter to roll dice).
Player 1 rolled a 2
```

```
Player 2's turn (Press Enter to roll dice).
Player 2 rolled a 5
Player 2 got bitten by a snake from 21 to 9!
Player 2 is now at position 9

Player 1's turn (Press Enter to roll dice).
Player 1 rolled a 1
Player 1 is now at position 29

Player 2's turn (Press Enter to roll dice).
Player 2 rolled a 2
Player 2 climbed a ladder from 11 to 26!
Player 2 is now at position 26

Player 1's turn (Press Enter to roll dice).
Player 1 rolled a 5
Player 1 needs exact number to reach 30!
Player 1 is now at position 29

Player 2's turn (Press Enter to roll dice).
Player 2 rolled a 1
Player 2 got bitten by a snake from 27 to 1!
Player 2 is now at position 1

Player 1's turn (Press Enter to roll dice).
Player 1 rolled a 1
Player 1 is now at position 30

Player 1 WINS THE GAME!
```

Q2) This system uses a dictionary as a hash table to perform fast insert, retrieve, and delete operations on student records.

```
@Farhan-Prog46 ➜ /workspaces/Data-Structures-and-Algorithms (main) $ /home/codespace/.python/current/bin/pytho
n /workspaces/Data-Structures-and-Algorithms/BTP500Group5-Assignment2/Question2_Syed_Farhan.py
Initial Records:
====== ALL STUDENT RECORDS ======
ID          Name            GPA
----------------------------------
101         Alice           3.6
102         Sara            2.5
103         John            3.0

Inserting a new student:
Student 'Farhan' added successfully.

Retrieving student with ID 101:
Student Record Found:
ID   : 101
Name : Alice
GPA  : 3.6

Deleting student with ID 102:
Student 'Sara' deleted successfully.

Final Records:
====== ALL STUDENT RECORDS ======
ID          Name            GPA
----------------------------------
101         Alice           3.6
103         John            3.0
104         Farhan          2.8
```

Q3) This program implements a Binary Search Tree to manage books and demonstrates insertion, search, and traversal methods.

```
@Farhan-Prog46 ➜/workspaces/Data-Structures-and-Algorithms (main) $ /home/codespace/.python/current/bin/pytho
n /workspaces/Data-Structures-and-Algorithms/BTP500Group5-Assignment2/Question3_Farhan_And_Umer.py

BST STRUCTURE
Root: Python Basics
            R--- Operating Systems
        R--- Database Systems
    L--- Data Structures
        L--- Algorithms

SEARCH RESULT
Book Found -> Title: Algorithms, Author: Michael Lee, Year: 2021

IN-ORDER TRAVERSAL
Title: Algorithms, Author: Michael Lee, Year: 2021
Title: Data Structures, Author: Alice Brown, Year: 2019
Title: Database Systems, Author: Sara Khan, Year: 2018
Title: Operating Systems, Author: David Miller, Year: 2022
Title: Python Basics, Author: John Smith, Year: 2020

PRE-ORDER TRAVERSAL
Title: Python Basics, Author: John Smith, Year: 2020
Title: Data Structures, Author: Alice Brown, Year: 2019
Title: Algorithms, Author: Michael Lee, Year: 2021
Title: Database Systems, Author: Sara Khan, Year: 2018
Title: Operating Systems, Author: David Miller, Year: 2022

POST-ORDER TRAVERSAL
Title: Algorithms, Author: Michael Lee, Year: 2021
Title: Operating Systems, Author: David Miller, Year: 2022
Title: Database Systems, Author: Sara Khan, Year: 2018
Title: Data Structures, Author: Alice Brown, Year: 2019
Title: Python Basics, Author: John Smith, Year: 2020
@Farhan-Prog46 ➜/workspaces/Data-Structures-and-Algorithms (main) $
```

Research Questions:

Q1) Completed by: Muhammad Umer – Group #5

Ans: In the snakes and ladders program, both a dictionary and a 1D list provide constant time lookup, which means both are efficient for accessing positions. However, a dictionary is more suitable for this problem because it only stores the positions where snakes and ladders exist. In contrast, a 1D list would store all board positions, even though most of them do not contain anything.

In terms of memory usage, a dictionary is more efficient because it avoids storing unnecessary positions, while a list uses more space by storing every position on the board.

In our implementation, the dictionary made it easier to check whether a player landed on a snake or ladder and update their position directly without extra checks.

Q2) Completed by: Syed Farhan – Group #5

Ans: In a student record system, using a hash table (dictionary) has several advantages over a sorted list.

The main advantage is direct access using a key. In a hash table, each student ID is used as a key, so the system can directly retrieve a student's record in O(1) time on average, without searching through other records.

In contrast, a sorted list requires searching. Even if we use binary search, which is faster than linear search, it still takes O(log n) time. Also, insertion and deletion in a sorted list are slower (O(n)) because elements may need to be shifted to maintain order.

Another advantage of a hash table is that insertion and deletion are also fast (O(1)), while in a sorted list they are more expensive.

In our implementation, we used a dictionary where the student ID is the key. This allows us to perform insert, retrieve, and delete operations instantly without looping through all records, making the system efficient and simple.

Q3) Completed by: Both – Group #5

Ans: In our BST program, I used three types of traversals, and each one has a different purpose.

- In-order traversal is used to display books in sorted order by title. This is useful when the library wants to show all books alphabetically to users. It visits the **left subtree -> root -> right subtree**
- Pre-order traversal is useful when we want to copy or recreate the structure of the tree, because it processes the root node first and preserves the structure of the tree**.** It visits the **root -> left subtree -> right subtree**
- Post-order traversal is useful when deleting or freeing the tree, because it processes child nodes before the parent, ensuring that all data is safely removed. It visits the **left subtree -> right subtree -> root**

Q4) Completed by: Both – Group #5

Ans: Time Complexities (Big-O)

For dictionaries:

- Insert → O (1)
- Search → O (1)
- Delete → O (1)

This is because dictionaries use hashing.
In rare worst cases, collisions can make operations slower, up to **O(n)**, but average performance is still very fast.

For BST:

Best / Average Case

When the tree is balanced:

- **Insert:** O(log n)
- **Search:** O(log n)
- **Delete:** O(log n)

Worst Case

When the tree becomes unbalanced, like a linked list:

- **Insert:** O(n)
- **Search:** O(n)
- **Delete:** O(n)

If the tree becomes skewed, it behaves like a linked list and becomes slower.

For traversals:

For all traversals:

- **In-order:** O(n)
- **Pre-order:** O(n)
- **Post-order:** O(n)

because every node must be visited once.

Since our BST is not self-balancing, its performance depends on the order of insertion. In our case, the tree is slightly unbalanced but still works efficiently for small data.

Q5) Completed by: Both – Group #5

Ans: The game can be improved in several ways.

- First, we ask the user how many players will play, then we can support multiple players by storing them in a list or dictionary and looping through them.
- Second, we can add a GUI using libraries like Tkinter or Pygame. This would make the game more interactive with a GUI, we can show:

I. The board layout
II. player positions
III. dice roll animation
IV. buttons instead of keyboard input
V. pop-up messages for snakes, ladders, and winners

- Third, we can add a save feature so the game can continue later. This can be done by saving player positions and game state in a file. This can be done by storing:

I. player names
II. player positions
III. turn number
IV. Board data

We can save this information in:

I. a text file
II. JSON file
III. database

Then, when the game starts again, the program can load the saved state and continue from where it stopped.

- Currently, the game runs in the terminal using input and print statements. Converting it to a GUI would improve user experience and make the game more realistic.

Team Contribution:

Question 1: Snakes and Ladders Game	Muhammad Umer - 178267233
Question 2: Hash Table – Student Records	Syed Farhan - 154714232
Question 3: Library System using BST	Muhammad and Syed